

# AI技术调研与场景分析

学号：2023141461061

姓名：袁新坪

课程项目：AI辅助软件过程改进平台

文档版本：V1.0

编写日期：2026年7月

## 摘要

随着大语言模型、代码智能分析、知识检索与自动化测试技术的发展，人工智能已经从单纯的内容生成工具逐步演变为可参与软件需求分析、设计评审、编码辅助、测试生成、缺陷定位、项目度量和过程改进的工程工具。本文围绕软件过程管理课程的实践目标，对生成式人工智能、检索增强生成、代码大模型、智能测试、过程挖掘和预测软件分析等技术进行调研，并结合“AI辅助软件过程改进平台”提出可落地的应用场景。

本文重点分析以下问题：AI可以在哪些软件生命周期活动中产生实际价值；不同场景对数据、模型、权限和可靠性的要求有何差异；如何避免模型幻觉、数据泄露和自动化误判；如何构建从试点、验证到规模化推广的实施路线。最终形成需求分析助手、代码评审助手、测试用例生成器、缺陷分析助手、知识问答助手和过程度量分析器六类核心场景，并通过价值、成本、风险和实施难度进行优先级排序。

**关键词：** 人工智能；软件过程改进；大语言模型；智能测试；代码评审；过程度量

## 1. 调研背景

### 1.1 软件过程中的主要问题

传统软件项目通常存在以下共性问题：

- 需求描述依赖自然语言，容易产生歧义、遗漏和不可验证内容。
- 代码评审高度依赖个人经验，评审标准不统一。
- 测试用例编写工作量大，边界条件和异常路径覆盖不足。
- 缺陷记录分散，根因分析多依赖人工复盘。

- 5. 项目文档、会议纪要、设计方案和历史问题难以快速检索。
- 6. 过程数据虽然大量存在于 Git、Issue、CI 和测试平台中，但缺少持续分析。
- 7. 团队在版本压力下容易牺牲规范，导致返工、延期和质量波动。

人工智能适合处理大量非结构化文本、代码、日志和历史记录，因此可以成为软件过程改进的重要支撑技术。但AI不能简单替代工程人员，而应嵌入现有流程，形成“机器建议、人员确认、过程留痕、持续度量”的协作模式。

## 1.2 调研目标

本次调研的目标包括：

- 梳理可用于软件过程改进的AI技术类别；
- 分析AI在需求、设计、开发、测试、运维和管理阶段的适用场景；
- 评估各场景的业务价值、数据条件、风险和实施难度；
- 为后续“软件过程改进方案设计”和“AI技术应用验证”提供依据；
- 形成可逐步实施的技术路线，而非一次性建设复杂平台。

## 1.3 调研范围

本次调研覆盖软件生命周期中的主要活动：

| 生命周期阶段 | 调研内容                |
|--------|---------------------|
| 项目启动   | 项目目标澄清、范围识别、风险提示    |
| 需求分析   | 需求抽取、歧义检测、验收标准生成    |
| 系统设计   | 架构建议、接口一致性检查、设计文档生成 |
| 编码实现   | 代码补全、规范检查、代码评审、重构建议 |
| 软件测试   | 测试用例生成、覆盖分析、缺陷预测    |
| 发布运维   | 日志分析、故障归因、变更风险分析    |
| 项目管理   | 进度总结、度量分析、知识沉淀、过程改进 |

## 2. AI技术分类调研

### 2.1 大语言模型

大语言模型能够理解和生成自然语言、代码、表格和结构化内容，适合处理需求文档、会议纪要、设计说明、代码评审意见和测试报告。

主要能力包括：

- 文本摘要与信息抽取；
- 需求重写与验收标准生成；
- 代码解释、补全和重构建议；
- 测试用例与测试数据生成；
- 缺陷描述规范化；
- 多文档知识问答。

局限性主要包括模型幻觉、上下文长度限制、对私有项目知识缺乏了解，以及输出内容需要人工复核。

### 2.2 检索增强生成

检索增强生成通过“先检索项目资料，再让模型回答”的方式降低幻觉。系统可以将需求文档、接口文档、编码规范、历史缺陷、测试报告和会议纪要建立索引，在用户提问时检索相关片段并提供给模型。

该技术适合以下场景：

- 查询某个业务规则的历史决策；
- 查找类似缺陷及其解决办法；
- 根据团队规范进行代码检查；
- 生成带来源依据的项目问答结果；
- 帮助新成员快速熟悉项目。

### 2.3 代码大模型与静态分析融合

代码大模型擅长理解代码语义，而静态分析工具擅长确定性规则检查。两者融合可以形成更可靠的代码评审机制：

- 静态分析负责语法错误、空指针风险、复杂度、重复代码和安全规则；
- 代码模型负责可读性、业务逻辑、命名、结构和潜在边界问题；
- 人工评审负责最终判断和架构层面决策。

这种组合比单独依赖AI更稳定，也比单独使用规则工具覆盖更广。

## 2.4 智能测试技术

智能测试包括基于需求生成测试用例、基于代码生成单元测试、基于接口描述生成接口测试、测试数据构造、失败日志分析和回归范围推荐。

AI在测试中的主要价值不在于完全替代测试人员，而在于：

- 快速生成测试初稿；
- 补充边界值、异常路径和组合场景；
- 将需求、接口和代码关联起来；
- 自动解释失败原因；
- 为回归测试选择提供建议。

## 2.5 过程挖掘与预测软件分析

过程挖掘通过分析事件日志还原真实流程。例如，根据需求创建、开发开始、代码提交、评审完成、测试通过和版本发布等时间点，可以识别等待时间、返工路径和流程瓶颈。

预测分析可利用历史数据预测：

- 需求延期风险；
- 缺陷高发模块；
- 变更可能影响的文件；
- 迭代剩余工作量；
- 发布失败概率。

此类技术对数据质量要求较高，适合作为中后期建设内容。

# 3. 项目场景说明

## 3.1 项目概述

“AI辅助软件过程改进平台”面向课程软件项目团队，目标是将需求、任务、代码提交、测试、缺陷和项目度量统一关联，并提供AI辅助能力。平台不直接替代现有代码仓库或项目管理工具，而是通过接口或数据导入方式获取过程数据。

核心用户包括：

| 角色     | 主要需求            |
|--------|-----------------|
| 项目负责人  | 掌握进度、风险、质量和改进效果 |
| 需求分析人员 | 提高需求完整性和可验证性    |
| 开发人员   | 获得编码、评审和缺陷修复建议  |
| 测试人员   | 快速生成测试用例并分析失败原因 |
| 质量保证人员 | 检查流程执行情况和度量指标   |
| 新加入成员  | 快速检索项目知识和历史决策   |

## 3.2 系统数据来源

平台计划使用以下数据：

- 需求文档与用户故事；
- 任务和缺陷记录；
- Git 提交信息、分支和合并请求；
- 代码评审意见；
- 单元测试、接口测试和系统测试结果；
- 构建与部署日志；
- 项目规范、模板和会议纪要；
- 迭代计划和复盘记录。

# 4. 核心AI应用场景

## 4.1 场景一：需求质量检查助手

### 目标

自动检查需求描述中的歧义、遗漏、冲突和不可验证表达，并生成建议的验收标准。

### 输入

- 用户故事；
- 功能需求说明；
- 业务规则；

- 原型说明；
- 历史同类需求。

## 输出

- 不明确术语列表；
- 缺失条件和异常场景；
- 可测试的验收标准；
- 风险提示；
- 建议的需求拆分结果。

## 示例

原始需求：

系统应快速完成用户登录，并在异常情况下给出友好提示。

AI检查后应指出：

- “快速”缺少可度量标准；
- “异常情况”范围不明确；
- 未说明连续登录失败后的处理；
- 未说明账号禁用、密码过期和网络异常；
- 建议定义响应时间、提示规则和安全限制。

## 4.2 场景二：AI代码评审助手

### 目标

在合并请求阶段自动分析代码变更，生成结构化评审建议。

### 检查维度

- 编码规范；
- 空值和异常处理；
- 资源释放；
- 安全风险；
- 性能问题；
- 重复代码；
- 业务规则遗漏；
- 测试覆盖情况；
- 变更影响范围。

约束

AI评审结果只作为建议，不能自动批准合并。高风险问题必须附带代码位置、原因和修改建议，评审人员确认后才可关闭。

4.3 场景三：测试用例生成助手

目标

根据需求、接口定义和代码变更生成测试用例初稿。

测试类型

- 正常流程测试；
- 边界值测试；
- 异常流程测试；
- 权限测试；
- 并发测试；
- 数据一致性测试；
- 回归测试；
- 安全性测试。

用例模板

| 字段   | 内容         |
|------|------------|
| 用例编号 | 自动生成       |
| 关联需求 | 需求编号       |
| 前置条件 | 执行前系统状态    |
| 输入数据 | 正常、边界或异常数据 |
| 操作步骤 | 可执行步骤      |
| 预期结果 | 可验证结果      |
| 优先级  | 高、中、低      |
| 测试类型 | 功能、接口、安全等  |

## 4.4 场景四：缺陷分析助手

### 目标

对缺陷进行分类、相似缺陷检索、根因提示和修复建议生成。

### 处理流程



缺陷助手重点解决“重复缺陷无法识别”“日志信息不足”“修复经验没有沉淀”等问题。

## 4.5 场景五：项目知识问答助手

### 目标

基于项目私有知识库回答问题，并给出引用来源。

典型问题包括：

- 某接口为何采用当前设计；
- 某字段的业务含义是什么；
- 上次类似故障如何解决；
- 当前项目的分支策略是什么；
- 发布前必须完成哪些检查；
- 某功能有哪些验收标准。

该场景应采用检索增强生成，避免模型凭空回答。

## 4.6 场景六：过程度量分析器

### 目标

分析项目过程数据并生成改进建议。

拟统计指标包括：

| 指标       | 说明              |
|----------|-----------------|
| 需求平均交付周期 | 从需求确认到验收完成的时间   |
| 代码评审等待时间 | 提交评审到首次反馈的时间    |
| 缺陷密度     | 单位模块或代码规模中的缺陷数量 |



| 指标      | 说明              |
|---------|-----------------|
| 缺陷重开率   | 已关闭缺陷再次打开的比例    |
| 构建成功率   | 自动构建成功次数占比      |
| 自动测试通过率 | 自动化测试通过比例       |
| 需求变更率   | 迭代中发生变更的需求比例    |
| 返工率     | 因遗漏或错误导致的重复工作比例 |

## 5. 场景优先级分析

采用价值、可行性、风险和数据准备度四个维度进行评分，每项满分5分。

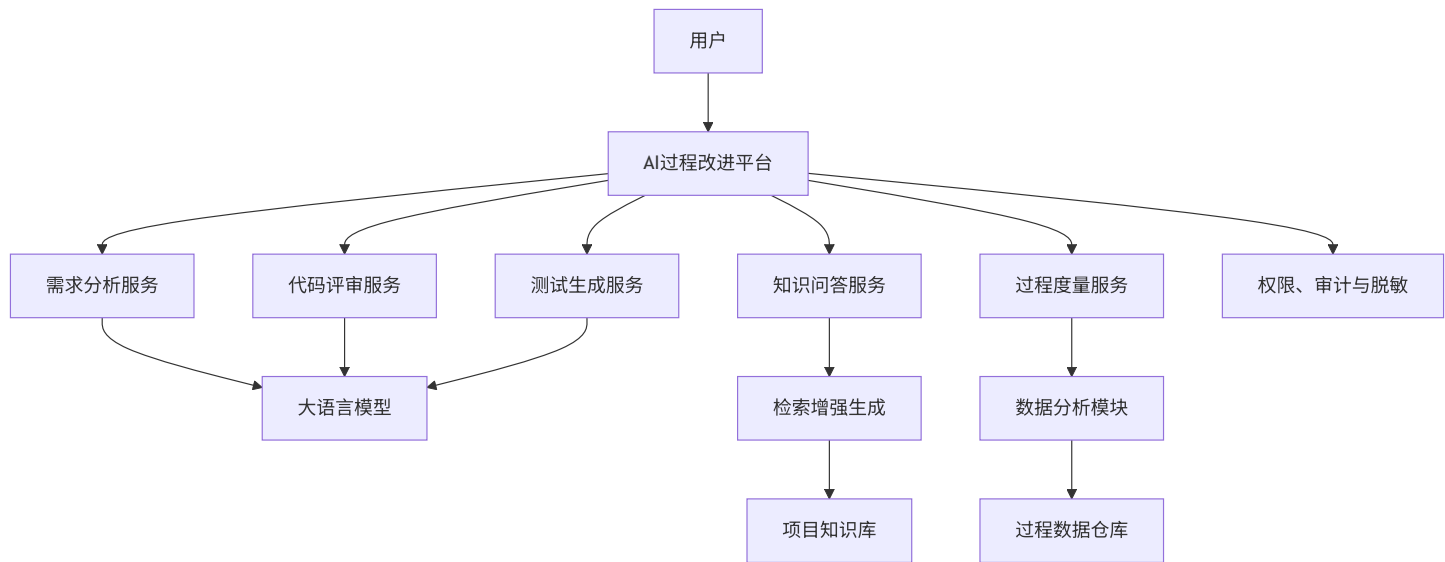
| 场景     | 业务价值 | 可行性 | 数据准备度 | 风险可控性 | 总分 |
|--------|------|-----|-------|-------|----|
| 需求质量检查 | 5    | 5   | 4     | 4     | 18 |
| 测试用例生成 | 5    | 4   | 4     | 4     | 17 |
| 项目知识问答 | 4    | 5   | 5     | 4     | 18 |
| AI代码评审 | 5    | 4   | 3     | 3     | 15 |
| 缺陷分析   | 4    | 4   | 3     | 4     | 15 |
| 过程度量分析 | 4    | 3   | 2     | 5     | 14 |

根据评分，建议第一阶段优先建设：

- 1. 需求质量检查助手；
- 2. 项目知识问答助手；
- 3. 测试用例生成助手。

第二阶段建设代码评审和缺陷分析，第三阶段在数据积累后建设过程挖掘与预测分析。

# 6. 技术架构设想



建议采用分层设计：

- **交互层：** 提供需求检查、评审、问答和报表界面；
- **业务层：** 管理提示模板、任务编排、结果审核和评价；
- **AI能力层：** 封装模型调用、检索、向量化和规则检查；
- **数据层：** 保存项目资料、过程数据、反馈和审计记录；
- **治理层：** 提供权限控制、脱敏、内容过滤和成本监控。

# 7. 风险分析

## 7.1 模型幻觉

AI可能生成不存在的接口、规则或历史结论。应通过项目知识检索、来源引用和人工确认降低风险。

## 7.2 数据泄露

项目代码、缺陷和用户数据可能包含敏感信息。应采用最小权限、传输加密、数据脱敏和私有化部署策略。

## 7.3 过度依赖

团队可能直接采用AI输出而不验证。应明确“AI输出为建议”，关键需求、架构、安全和发布决策必须由人员负责。

## 7.4 评价偏差

若只追求生成数量，可能导致大量低质量需求或测试用例。评价指标应关注采用率、缺陷发现率、返工减少和时间节省。

## 7.5 成本和性能

模型调用会带来费用与响应延迟。应通过缓存、模型分级、批量处理和使用限额控制成本。

# 8. 实施路线

### 第一阶段：原型验证

- 选择需求检查和测试用例生成两个场景；
- 准备少量脱敏数据；
- 建立统一输入输出模板；
- 记录人工完成时间和AI辅助时间；
- 评价准确性、可用性和采用率。

### 第二阶段：流程集成

- 接入需求管理和代码仓库；
- 在合并请求中增加AI评审；
- 建立人工确认机制；
- 保存AI建议、采纳结果和修正内容；
- 形成可追溯记录。

### 第三阶段：知识与度量

- 建设项目知识库；
- 关联需求、代码、测试和缺陷；
- 建立过程指标看板；
- 使用历史数据识别瓶颈；
- 持续优化提示模板和规则。

# 9. 预期效益

预期效益不应只用“生成速度快”衡量，而应从过程质量和工程结果进行评价。

| 目标   | 预期改善           |
|------|----------------|
| 需求质量 | 减少模糊需求和验收标准遗漏  |
| 评审效率 | 缩短常规问题发现时间     |
| 测试覆盖 | 增加边界和异常场景      |
| 缺陷处理 | 加快相似问题检索与根因定位  |
| 知识传递 | 降低新成员学习成本      |
| 过程管理 | 提高数据透明度和改进决策依据 |

# 10. 结论

AI在软件过程改进中的价值主要体现在信息理解、知识检索、内容生成、模式识别和决策辅助。对于课程项目，最适合的实施方式不是直接建设复杂的全自动系统，而是从高频、低风险、可验证的场景开始，通过人工确认和数据度量逐步扩大范围。

本次调研认为，需求质量检查、项目知识问答和测试用例生成具有较高的优先级；代码评审、缺陷分析和过程挖掘适合作为后续扩展。所有场景都必须遵循数据安全、输出可追溯、人员最终负责和持续评价的原则。

# 参考资料

- ISO/IEC/IEEE 12207：软件生命周期过程。
- CMMI Development：软件过程能力与持续改进相关实践。
- SWEBOK：软件工程知识体系。
- OWASP：应用安全与安全开发实践。
- 软件需求工程、软件测试和软件质量保证相关课程资料。